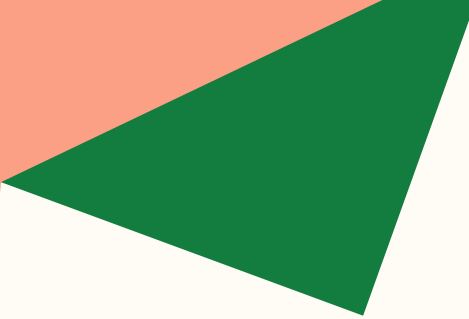




Retrieval Augmented Generation (RAG)

Dr. Abhishek Grover
Assistant Professor
Communication and Computer Engineering, LNMIIT

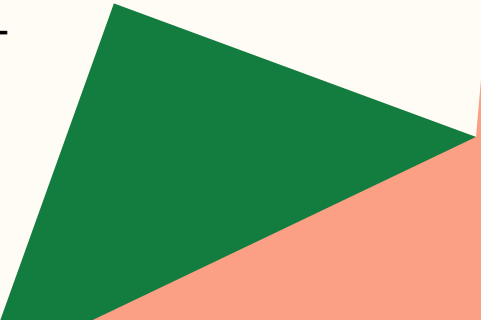


Table of Contents

3 On-the-fly Prompt

5 Context Window Constraint

7 Search as an Optimization

8 Need for Vector Databases

9 Ingestion

11 Logic of Chunking

12 Vector Embeddings

13 Retriever

14 Implementation

13 Retriever

On-the-Fly Prompt

- Prompt engineering is a way to steer the agent in a specific direction.
- Giving relevant data in context window simplifies the steering mechanism.
- LLM has undergone SFT.
- SFT ensures that the agent's output is primarily dependent on the context window.

On-the-Fly Prompt

- Therefore giving documents in LLM's context window is a good way to narrow down LLM's vision.
- **Parametric Memory:** Knowledge stored in the model's weights (long-term, hard to update)
- **Non-Parametric Memory:** External data provided in the prompt (short-term, easily updated)
- RAG uses Non-parametric memory to enable In-context learning.

Context Window Constraint

- **Finite Token Limit:** Every model has hard architectural limit on context window length (eg. 8k,32k,128k).
- **Quadratic complexity:** Self-Attention mechanism has computational and memory complexity of $O(n^2)$.
- Doubling the input length quadruples the resource requirement.
- **Video RAM (VRAM) and KV Cache:** Storing Key-Value (KV) Cache for long sequences consume massive amounts of GPU memory.

Context Window Constraint

- **Lost in the Middle Phenomenon:** Many studies have shown that LLM performance degrades when relevant information is buried in the middle of a long context.
- Models tend to prioritize the beginning (primacy) and end (recency) of the provided text.
- **Noise:** Filling the context window with irrelevant documents decreases signal-to-noise ratio which leads to errors.
- **Latency:** Long context lead to higher latency and higher API costs.

Search as an Optimization

- **The Problem($O(N)$ Linear Scan)**: As corpus grows, the cost of "scan" becomes computationally prohibitive.
- **Optimization**: Transform a corpus of size D into a manageable subset k that can be filled in context window.
- **Keyword Filtering (BM25)**: Filtering based on exact term frequency.
- **Semantic Filtering**: Filtering based on latent meaning which ensures context of filtered documents is relevant to input query.
- Semantic filtering implemented using **Vector Databases**.

Need for Vector Databases

- **Limitation of Relational DBs:** Traditional B-tree or Hash-based indexes are designed for exact matches (e.g., WHERE id=10) or 1D range queries.
- **From equality to Proximity:** Vector-Databases are purpose-built to perform Nearest Neighbour (NN) searches in spaces with 768 to 1356+ dimensions.
- **Storage:** A Vector DB stores the mathematical representation (the embedding) alongside the raw text/metadata allowing the system to “query” meaning rather than strings.
- **Search capability:** Allow for execution of vector similarity of input query with stored data.

Ingestion

- **“ETL” for LLM:** Ingestion is Extract, Transform, and Load process that converts raw, unstructured data into a structured, queryable vector format.
- **Step 1: Data Extraction and Parsing:** Normalizing diverse sources (PDFs, HTML, Markdown, SQL exports) into a unified text stream.
- **Step 2: Chunking:** Breaking continuous text into discrete segments.
- Too small chunks may lead to loss in context.
- Large chunks may dilute the semantic signal.

Ingestion

- **Step 3: Vectorization (Embedding):** Pass chunks through an Encode model (BERT, RoBERTa) to generate high dimensional vectors (v in R^d).
- **Step 4: Data Loading:** Loading the vectors and their corresponding text/metadata into the vector database index for retrieval.
- **Index Structure:** The database organizes data so that “neighbours” can be found without checking every record.
- **Example strategy:** The vector database maintains k centroids. A new chunk will be indexed in database based on its distance from centroid.

Logic of Chunking

- **"Goldilocks" Problem:** Chunks cannot be very small and cannot be very large.
- **Overlapping:** A 10%-20% overlap between chunks should be ensured.
- **Recursive character splitting:** It is good to split at logical boundaries such as double newlines, single newlines and spaces.
- **Document-based splitting:** Using document's own structure (Markdown headers, Latex sections or HTML tags) to define boundaries.
- **Semantic chunking:** Keep semantically related sentences in a single chunk.

Vector Embeddings

- **Embedding function:** A transformer based encoder (BERT, RoBERTa) maps a discrete text chunk T into a dense, continuous vector space of fixed dimensionality (eg. 768, 1536).
- **Similarity:** Chunks are said to similar if they are close in embedding space.
- **Normalization:** Embeddings are often normalized to unit vectors.
- **Directionality:** The direction (angle) of the vector is more representative of meaning.

Retriever

- **Preprocessing:** Vectorization of user's query.
- **Search:** Execute the centroid based search
- **Filtering:** Applying boolean filters (eg., Find matches where author='expert' or domain='finance')
- **Ranking:** Returning the sorted Top-k results based on the similarity metric.

Implementations

- **Managed Services (Saas):** Interact with database using an API to “upsert” and “query” vectors without managing servers, indexes or sharding. Eg. Pinecone, MongoDB Atlas Vector Search.
- **Open Source (Self hosted):** Deploy the database on you machine. Full control over storage engine and clustering. Eg., Milvus, Weaviate, Quadrant, Chroma.
- **Database Extensions (Hybrid):** Adding vector capabilities to a database. Eg., pgvector (Postgre SQL).